

HapTest动态分析框架的事件生成策略优化研究

——基于UI组件优先级的自适应事件调度

开题报告

一、研究意义

1.1 选题背景

OpenHarmony作为我国自主研发的国产全场景分布式操作系统，自发布以来呈现出爆发式增长态势。截至2024年，OpenHarmony生态已累计超过15,000款应用，搭载设备突破9亿台，覆盖智能手机、智慧屏、车载系统等多类IoT设备。OpenHarmony凭借其强大的跨平台能力和分布式架构，已成为Gitee平台上最活跃的开源项目之一。随着鸿蒙生态的快速扩张，应用质量保障成为制约生态健康发展的关键瓶颈。

ArkTS是专为OpenHarmony应用开发设计的新型编程语言。与Android应用类似，采用ArkTS开发的鸿蒙应用同样面临崩溃、性能瓶颈、安全漏洞等质量问题。动态分析是检测运行时错误、性能瓶颈和安全漏洞的重要手段——通过在程序运行时收集执行信息来分析程序行为，能够捕获静态分析难以触及的运行时行为。然而，截至HapTest提出之前，开发者社区尚没有一个专门针对OpenHarmony应用动态分析的框架。

为了量化“工具缺失”的程度，相关研究对Android和OpenHarmony的研究投入进行了对比。在DBLP数据库中，以“Android”为关键词可检索到超过7,000篇论文，而OpenHarmony相关论文仅有8篇。这种近千倍的差距直观地反映了OpenHarmony软件工程研究的空白状态，也意味着巨大的研究机遇。

HapTest是首个专门针对OpenHarmony应用设计的动态分析框架，由Liu等人于2025年提出，发表于CCF A类会议FSE 2025（第33届ACM国际软件基础研讨会）。该框架通过代码插桩（CIM）、事件生成（EGM）、执行监控（EM）和数据处理（DPM）四个模块，实现了对鸿蒙应用的自动化测试与动态分析。HapTest已作为开源项目公开发布。实验证明，HapTest在OpenHarmony应用市场排名前20的热门商业应用（每个应用至少百万级下载量）中，成功发现了26个此前未报告的崩溃问题，涉及11款应用。

然而，HapTest的事件生成模块（EGM）目前仅支持贪心DFS（深度优先搜索）、BFS（广度优先搜索）、随机探索等固定策略。策略一旦选定即固定不变，缺乏对UI组件语义的理解和优先级区分，导致大量测试事件耗费在低价值路径上，测试效率有待进一步提升。HapTest的论文也明确指出，框架的组件可被进一步定制以实现特定的动态分析需求——本研究正是对这一开放性的积极响应。

1.2 研究意义

理论意义：本研究探索在动态分析框架的事件生成环节引入UI组件优先级感知机制，将组件语义信息与覆盖率反馈相结合，实现更智能的事件调度。这一研究思路丰富了移动应用自动化测试的理论体系，为鸿蒙生态的测试工具研究提供了新的方法论参考。

实践意义：通过优化HapTest的EGM模块，使其能够根据UI组件的语义重要性和实时覆盖率反馈动态调整事件生成策略，预期可显著提高测试效率和缺陷检出率。这将帮助开发者更高效地保障应用质量，降低测试时间成本，直接服务于鸿蒙生态建设。

二、国内外研究现状

2.1 移动应用动态分析技术现状

动态分析是软件测试领域的重要技术手段，通过在程序运行时收集执行信息（如覆盖率、性能数据、运行时状态）来分析程序行为。在Android生态中，学术界和工业界已积累了丰富的研究成果。

自动化测试工具的演进：早期的Monkey工具采用纯随机事件生成，实现简单但覆盖率低下。随后出现的模型驱动方法（如MobiGUITAR）通过构建GUI状态机进行系统探索，但模型构建和维护成本较高。搜索驱动方法（如Sapienz）采用多目标搜索算法优化测试序列，在Android上取得了显著成效。近年来，AI测试利用大语言模型（LLM）理解应用语义，智能生成测试场景，成为前沿方向。

然而，这些工具的核心假设——基于Activity的生命周期模型和传统Java/Dalvik执行环境——与鸿蒙的Stage模型和ArkTS语言存在根本性差异，无法直接迁移。

测试用例优先级排序技术：在软件测试领域，测试用例优先级排序已被证明能有效提高回归测试的效率。多目标测试用例优先级排序问题旨在根据多个测试标准对测试用例进行排序。已有研究将测试用例优先级排序建模为多目标优化问题，采用进化搜索算法进行求解。在UI测试中，有研究提出了以网页元素为目标的覆盖标准，作为优化问题的目标函数。这些研究成果为本文提出的基于UI组件优先级的事件调度策略提供了理论参考。

2.2 鸿蒙应用质量保障研究进展

鸿蒙生态的质量保障研究尚处于起步阶段。根据Li等人发表的OpenHarmony软件工程研究路线图，目前该领域存在显著的研究空白。

HapTest框架：2025年，Liu等人提出了首个面向OpenHarmony的动态分析框架HapTest。该框架的核心贡献包括：

- **模块化架构：**CIM（代码插桩）、EGM（事件生成）、EM（执行）、DPM（数据处理）四模块设计，支持策略可插拔
- **PTG（页面转换图）：**解析UI层次结构，动态构建页面跳转关系，指导事件生成
- **DataHub：**结构化存储运行时数据，支持策略优化

- **有效性验证：**在10个开源应用上实现40%-90%的行覆盖率，在20个头部商业应用中发现了26个未知崩溃

EGM模块的现状：EGM提供一个策略适配器，采用四阶段流水线，支持贪心DFS、随机探索策略。然而，现有策略均为固定策略——策略在测试启动时选定，执行过程中不与DPM进行实时数据交互，无法根据测试进展和组件特性动态调整。

ArkAnalyzer静态分析：Chen等人开发的ArkAnalyzer是首个针对ArkTS语言的静态分析框架。后续的APAK（ArkAnalyzer Pointer Analysis Kit）作为首个上下文敏感的指针分析框架，已被合并入ArkAnalyzer。

2.3 ArkUI组件系统与UI测试技术

ArkUI组件树：OpenHarmony的ArkUI框架提供了两种主要的UI组件开发范式——ArkTS声明式组件和NDK原生组件。组件遵循树形结构，每个组件可以包含子组件。组件是构建页面的核心，每个组件通过对数据和方法的简单封装，实现独立的可视、可交互功能单元。组件类型包括基础组件（Text、Image、Button、TextInput等）、容器组件（Column、Row、Stack、List、Grid等）和导航组件（Navigation、NavDestination、Tabs等）。

ArkUI Inspector是DevEco Studio中集成的布局分析工具，开发者可以在组件树上选择组件，UI界面自动框选对应组件，属性列表显示当前组件的属性信息。

组件优先级机制：在OpenHarmony的ArkUI框架中，组件具有displayPriority属性，用于设置在布局容器中显示的优先级。当子组件的displayPriority值大于1时，数值越大优先级越高；若父容器空间不足，低优先级的子组件会被隐藏。这一机制为测试场景中组件优先级的划分提供了参考思路——在UI自动化测试中，可以借鉴类似的思想，为不同类型的UI组件分配不同的测试优先级。

UI自动化测试策略：在UI自动化测试实践中，优先覆盖高频、高风险的业务场景是公认的有效策略。采用Page Object Model设计模式，将UI元素封装为独立对象，可提升代码可维护性和复用性。这些实践为本研究提出的基于组件优先级的事件调度策略提供了工程层面的支撑。

三、HapTest框架概述与现有问题分析

3.1 HapTest框架架构

HapTest采用模块化架构，由四个核心模块组成：

模块	全称	核心职责
CIM	代码插桩模块	对OpenHarmony应用的ArkTS源码进行代码插桩
EGM	事件生成模块	提供策略适配器，采用四阶段流水线生成测试事件
EM	执行模块	控制应用运行、模拟用户交互并收集运行时数据

模块	全称	核心职责
DPM	数据处理模块	将采集到的数据进行处理，构建PTG，存储DataHub

核心数据流：CIM插桩 → EGM生成事件序列 → EM执行事件 → DPM收集数据（覆盖率/PTG/日志）。

HapTest具备多项基础动态分析能力（如PTG、DataHub等），可供开发者复用，并可进一步定制以实现特定的动态分析需求（如检测恶意软件或性能问题）。

3.2 EGM模块的局限性

通过对HapTest论文和专利文献的研读，梳理出EGM模块当前存在以下局限：

问题一：组件无差别对待。当前EGM对所有UI组件一视同仁——登录按钮、支付按钮与返回按钮、帮助按钮具有相同的被选中概率。大量测试事件耗费在低价值、低风险的UI路径上，而高风险路径（如支付流程、权限申请）未能得到充分探索。

问题二：缺乏反馈机制。EGM虽然在策略适配器层面支持结合PTG与DataHub数据动态调整事件生成逻辑，但目前的实现中策略在测试启动时选定，执行过程中与DPM的实时数据交互有限，无法根据已获得的覆盖率信息动态调整后续事件生成方向。测试后期大量事件重复执行在已覆盖路径上。

问题三：策略固定。虽然支持DFS、BFS、随机等多种策略，但策略一旦选定即固定不变，无法根据应用特点和测试进展动态切换或调整。

3.3 改进的必要性

HapTest已在商业应用中验证了其有效性，但EGM模块的策略优化空间意味着框架的测试效率仍有提升潜力。在UI自动化测试中，优先覆盖高频、高风险的业务场景是公认的有效策略——将这一思路引入HapTest的事件生成环节，有望在保持原有覆盖率水平的前提下，显著减少达到相同覆盖率所需的事件数，从而提升测试效率。

四、改进方案：基于UI组件优先级的自适应事件调度

4.1 改进目标

在HapTest的EGM模块中引入**UI组件优先级感知机制**和**覆盖率反馈驱动的动态权重调整**，使事件生成能够：

- 1. 优先探索高风险、高价值的UI组件（如登录按钮、支付按钮、权限请求弹窗）
- 2. 根据实时覆盖率数据动态调整探索方向
- 3. 在保证探索多样性的前提下提高测试效率

4.2 总体架构设计

在EGM中新增三个子模块，形成“识别-计算-生成”的处理流水线：

（一）组件解析器（Component Parser）

在CIM插桩阶段扩展能力，从ArkUI组件树中提取可交互组件信息。OpenHarmony的ArkUI框架中，组件遵循树形结构，组件类型包括基础组件（Button、TextInput等）、容器组件（List、Grid等）和导航组件（Navigation、Tabs等）。

组件解析器的具体工作包括：

- 组件类型识别：**识别Button、TextInput、List、Slider、Dialog等组件类型
- 组件层级获取：**记录组件在UI树中的位置和嵌套关系
- 组件标识建立：**为每个可交互组件建立唯一标识，关联到代码位置

组件类型可通过ArkUI框架的组件属性进行识别，也可参考ArkUI Inspector的组件树展示方式进行验证。

（二）优先级计算引擎（Priority Engine）

设计组件的优先级计算公式：

$$P_i = P_{base}(type_i) \times (1 + \alpha \times \frac{1}{cov_i + \epsilon})$$

其中各参数含义如下：

- $P_{base}(type_i)$ ：基于组件类型的基础优先级，通过预定义的优先级映射表确定
- cov_i ：该组件关联代码的当前覆盖率（从DPM获取，取值0~1）
- α ：覆盖率反馈强度系数（可配置，建议初始值0.5）
- ϵ ：小常数（如0.01），防止除零

基础优先级映射表（示例，可通过配置文件调整）：

优先级等级	优先级值	组件类型	典型示例	理由
最高	3	提交类按钮、支付组件	登录按钮、支付确认、订单提交	直接关联核心业务逻辑，崩溃风险高
最高	3	权限请求弹窗	相机权限、位置权限、通讯录权限	涉及敏感API，是安全检测的重点
高	2	导航组件	页面跳转按钮、Tab切换	影响页面覆盖率，决定PTG完整性

优先级等级	优先级值	组件类型	典型示例	理由
高	2	列表项	商品列表、 联系人列表	数据驱动UI， 遍历可覆盖大量代码
中	1.5	输入组件	文本框、搜索框	输入事件可触发数据验证逻辑
中	1	滑动组件	Slider、 ScrollView	触发滚动加载等逻辑
低	0.5	辅助功能组件	返回按钮、 帮助按钮、 设置入口	对核心功能覆盖贡献有限

动态调整机制：

- 每执行N个事件后（N可配置，建议10-20），从DPM查询最新覆盖率数据
- 对每个组件重新计算 P_i ，更新采样权重
- 若某组件关联代码覆盖率已达100%，将其权重降至接近0，避免重复探索
- 若某组件长时间未被选中，适当提升其权重，避免“饿死”

（三）事件采样器（Event Sampler）

改造EGM的事件生成逻辑，从固定策略改为按权重采样：

- **ϵ -greedy策略**：以概率 ϵ （建议0.1-0.2）随机选择组件（保持探索多样性，避免陷入局部最优），以概率 $1-\epsilon$ 按权重 P_i 采样组件（利用反馈信息优先探索高价值路径）
- **事件序列生成**：每次采样一个组件后，根据组件类型生成对应的交互事件——点击按钮、输入文本、滑动列表等
- **策略回退**：保留原始DFS/BFS/随机策略的调用接口，支持切换和对比实验

（四）与DPM的反馈闭环

建立EGM与DPM之间的轻量级数据交互：

- EGM定期向DPM查询覆盖率摘要（语句覆盖率、分支覆盖率、页面覆盖率）
- DPM返回各组件关联代码的覆盖状态
- EGM根据反馈更新优先级权重

这一反馈闭环正是对HapTest“组件可被进一步定制以实现特定动态分析需求”这一设计理念的具体实践。

4.3 实施步骤

第一阶段：代码研读与设计

- 深入研读HapTest源码，重点理解EGM的事件生成循环和DPM的数据结构
- 设计组件解析器、优先级引擎、事件采样器的接口和数据结构
- 定义组件类型识别规则和基础优先级映射表

第二阶段：核心功能开发

- 扩展CIM插桩，增加组件树信息采集能力
- 实现组件解析器，从插桩数据中提取可交互组件列表
- 实现优先级计算引擎，集成DPM覆盖率查询接口
- 改造事件采样器，实现 ϵ -greedy按权重采样

第三阶段：集成测试与对比实验

- 在2-3个开源鸿蒙应用上部署改进后的HapTest
- 分别运行原始版本和改进版本，记录测试数据
- 对比指标：达到相同覆盖率所需事件数、最终覆盖率、崩溃发现数量

第四阶段：结果分析与总结

- 分析实验数据，评估改进效果
- 撰写结题报告，总结技术方案和经验教训

五、可行性分析

5.1 技术可行性

模块化架构支持：HapTest采用模块化设计，EGM与DPM之间已有数据交互基础，为插入优先级计算和反馈查询提供了天然接口。

插桩能力可扩展：CIM已具备对ArkTS源码进行插桩的能力，可在此基础上扩展组件树信息的采集。

组件信息可获取：OpenHarmony的ArkUI框架中，组件遵循树形结构，组件类型可参照ArkUI官方文档的分类进行识别。ArkUI Inspector工具也提供了组件树的可视化查看能力，验证了组件信息获取的可行性。

算法复杂度可控：优先级计算为简单的加权公式（ $O(n)$ 复杂度），事件采样为按权重随机选择（ $O(n)$ 复杂度），不会对测试执行造成显著的性能开销。

5.2 资源可行性

- HapTest已开源，代码可获取
- 技术文档和实验环境完备
- 有开源鸿蒙应用可供实验验证
- 开发环境（Python/Node.js/HDC工具链）可搭建

5.3 风险与应对

风险	影响程度	应对措施
组件识别准确率不足	中	采用多特征（组件类型、ID、文本内容）融合识别；允许通过配置文件人工修正组件分类
优先级计算带来性能开销	低	采用轻量级算法；采样更新频率可配置（如每20个事件更新一次）
策略可能陷入局部最优	中	保留 ϵ -greedy中的随机探索因子（ $\epsilon \geq 0.1$ ），保证探索多样性
与HapTest原架构的兼容性	低	采用策略模式，原始策略作为可选项保留，支持一键回退对比

六、进度安排与预期成果

6.1 研究目标

- 1. 深入理解HapTest框架的源码实现，特别是EGM模块的事件生成机制和DPM的数据结构
- 2. 设计并实现基于UI组件优先级的自适应事件调度方案
- 3. 通过对比实验验证改进效果（预期达到相同覆盖率所需事件数减少20%-30%）
- 4. 撰写结题报告，将改进代码开源

6.2 进度安排

阶段	时间	主要工作内容	产出物
第一阶段	2026.06	研读HapTest论文和源码，重点理解EGM事件生成算法和CIM插桩机制；调研ArkUI组件树结构	开题报告、技术调研笔记
第二阶段	2026.06	设计组件解析器和优先级引擎接口；扩展CIM插桩能力；实现核心代码	改进版原型代码
第三阶段	2026.07	在开源鸿蒙应用上执行对比实验，收集覆盖率数据和事件效率数据	实验报告、对比图表
第四阶段	2026.07	撰写结题报告，整理代码并上传GitHub	结题报告、项目仓库

6.3 预期成果

- 1. **改进版HapTest框架**：EGM模块增强，具备基于UI组件优先级的自适应事件生成能力
- 2. **对比实验报告**：包含覆盖率增长曲线、事件效率对比等定量数据
- 3. **结题报告**：系统阐述设计方案、实现细节、实验结果与经验教训
- 4. **代码开源**：将改进代码整理后上传至GitHub